

PlastiWatch — IMU Data Collector

Embedded firmware for the **Seeed Studio XIAO ESP32C3** that collects 6-axis motion data from an **MPU6050** sensor, labels it with an activity name, and uploads it wirelessly to **Edge Impulse** for TinyML model training.

```
Physical Motion → MPU6050 → ESP32C3 → WiFi → Edge Impulse Training Dataset
```

Table of Contents

- 1. [Quick Start](#)
- 2. [What You Need](#)
- 3. [Hardware Wiring](#)
- 4. [Software Setup](#)
- 5. [Configure the Firmware](#)
- 6. [Build and Flash](#)
- 7. [Collecting Your First Data](#)
- 8. [Understanding the Display](#)
- 9. [Serial Monitor Commands](#)
- 10. [LED Status Guide](#)
- 11. [Troubleshooting](#)
- 12. [Code Deep Dive](#)

Quick Start

Follow these five steps to go from unboxed hardware to uploading your first dataset:

```
Step 1 → Wire up the hardware      (see Hardware Wiring)
Step 2 → Edit Config.h             (WiFi credentials + Edge Impulse API key)
Step 3 → Build and flash           (one click in VS Code / PlatformIO)
Step 4 → Power on the device       (watch the OLED boot screen)
Step 5 → Press button to pick label, then long-press (2 s) or type 'start' to record
```

That's it. The device records 500 IMU samples and uploads them automatically.

What You Need

Hardware

Item	Notes
------	-------

Item	Notes
Seed Studio XIAO ESP32C3	The microcontroller board
MPU6050 IMU module	6-axis (accelerometer + gyroscope)
SSD1306 OLED display (128×64)	I2C interface
Tactile push button	Momentary, any size — short press cycles label; long press starts recording
Jumper wires	Female-to-female for breadboard
Breadboard	Optional but recommended
USB-C data cable	Must support data — charge-only cables won't work

Software

Tool	Link
VS Code	https://code.visualstudio.com
PlatformIO IDE extension	Search "PlatformIO IDE" in VS Code Extensions
Edge Impulse account (free)	https://studio.edgeimpulse.com

Driver note — Windows: If the XIAO's serial port does not appear, install the **CH343** driver. Search for **CH343SER.EXE** on the Seeed Wiki and run the installer.

Hardware Wiring

Both the MPU6050 and the OLED share the **same I2C bus** (SDA + SCL lines). You can connect them in parallel — all three components share the same two wires. The label button requires no external resistor because the firmware enables the ESP32-C3's **internal pull-up resistor** on GPIO3.

XIAO ESP32C3

3.3V ————— VCC

GND ————— GND

D4 (GPIO6 / SDA) ————— SDA

D5 (GPIO7 / SCL) ————— SCL

MPU6050

VCC ————— VCC

GND ————— GND

SDA ————— SDA

SCL ————— SCL

AD0 — GND (sets I2C address to 0x68)

OLED SSD1306

VCC ————— VCC

GND ————— GND

SDA ————— SDA

SCL ————— SCL

Label / record button (no external resistor needed)

D3 (GPIO3) ————— Button leg A

GND ————— Button leg B

When button is OPEN: GPIO3 is pulled HIGH by internal resistor → not pressed

When button is PRESSED: GPIO3 is pulled LOW by GND → press detected

Pin Reference Table

XIAO ESP32C3 Pin	Connects to
3.3V	MPU6050 VCC and OLED VCC
GND	MPU6050 GND and OLED GND and button leg B
D3 (GPIO3)	Button leg A — short press cycles label; hold 2 s starts recording
D4 (GPIO6)	MPU6050 SDA and OLED SDA
D5 (GPIO7)	MPU6050 SCL and OLED SCL
(MPU6050 AD0)	GND — sets I2C address to 0x68

Power warning: The MPU6050 runs on **3.3 V**. Do not connect VCC to 5 V — it will damage the sensor.

Why no resistor? The ESP32-C3 has built-in pull-up resistors (~45 kΩ) on every GPIO pin. Enabling `INPUT_PULLUP` in firmware connects this resistor internally, so the pin reads HIGH when the button is open and LOW when pressed. Adding an external pull-up resistor is unnecessary.

Software Setup

1. Install VS Code and PlatformIO

1. Download and install [VS Code](#)
2. Open VS Code → Click the **Extensions** icon in the left sidebar (or `Ctrl+Shift+X`)
3. Search for "**PlatformIO IDE**" and click **Install**
4. Restart VS Code when prompted
5. Confirm the **PlatformIO alien-head icon** appears in the left sidebar

2. Open the Project

File → **Open Folder** → select the `plastiwatch_collector` folder.

PlatformIO detects `platformio.ini` automatically. You will see `seed_xiao_esp32c3` in the bottom status bar.

3. Set Up an Edge Impulse Project

1. Create a free account at studio.edgeimpulse.com
2. Click **Create new project** and name it (e.g. "PlastiWatch Activity")
3. Go to **Dashboard** → **Keys** and copy your **API key** — you will need it next

Configure the Firmware

Open `src/config/Config.h` and update these four settings before flashing:

```
// — 1. Your WiFi network —————
#define WIFI_SSID      "YourNetworkName"
#define WIFI_PASSWORD  "YourPassword"

// — 2. Your Edge Impulse API key (Dashboard → Keys) —————
#define EI_API_KEY     "ei_XXXXXXXXXXXXXXXXXXXXXXXXXXXX"

// — 3. A unique name for this device —————
#define EI_DEVICE_NAME "plastiwatch-001"

// — 4. Activity labels (must match your Edge Impulse project labels) —————
static const char* const ACTIVITY_LABELS[NUM_ACTIVITY_LABELS] = {
    "standing",
    "walking",
    "running",
    "falling"
};
```

Important: The XIAO ESP32C3 supports **2.4 GHz Wi-Fi only**. Make sure your router broadcasts a 2.4 GHz network.

Everything else in `Config.h` has sensible defaults (100 Hz sample rate, 5-second windows) and can be left unchanged for your first session.

Build and Flash

Option A — VS Code Status Bar (Recommended)

Look at the bottom status bar in VS Code for these buttons:

✓ Build Upload  Monitor

1. Click ✓ **Build** to compile (watch the terminal — it should end with **SUCCESS**)
2. Plug in the XIAO via USB-C
3. Click **Upload** to flash the firmware
4. Click  **Monitor** to open the serial console at 115200 baud

Option B — Command Line

```
# Compile (release build)
pio run -e seeed_xiao_esp32c3

# Compile and upload
pio run -e seeed_xiao_esp32c3 --target upload

# Open serial monitor
pio device monitor -b 115200
```

Build Environments

Environment	Debug logging	When to use
seed_xiao_esp32c3	Off	Normal data collection sessions
seed_xiao_esp32c3_debug	On (verbose)	Development and troubleshooting

If the Board Won't Enter Programming Mode

Hold **BOOT**, press and release **RESET**, then release **BOOT** to force the bootloader. Then run the upload command again.

Collecting Your First Data

Boot Sequence

On power-up, the OLED shows the splash screen and the serial monitor prints:

```
=====
PlastiWatch - IMU Data Collector  v1.0
=====
[INF][Main      ] Initialising IMU...
[INF][IMUSensor ] Initialised - accel ±2g  gyro ±250dps  sample rate 100Hz
[INF][Main      ] Connecting to WiFi...
[INF][Network   ] Connected - IP: 192.168.1.42  RSSI: -58 dBm
[INF][Collector ] State: IDLE   Label: "standing"  WiFi: connected

Serial commands: type "help" for a full list.
Button (GPIO3): short press = cycle label | hold 2 s = start recording
Recording      : also startable via serial command start
```

Step-by-Step: Collecting One Labeled Window

- ①

Power on the device – the IDLE screen appears on the OLED.
- ②

Press the label button (GPIO3) briefly to cycle through the activity labels:
standing → walking → running → falling → standing → ...
The label updates on the OLED immediately after each press.
- ③

Select the correct label for the motion you are about to perform.
- ④

Start recording – choose either method:
 - Hold the button for 2 seconds (release when the OLED switches to RECORDING)
 - Or type `start` in the serial monitor and press EnterThe LED blinks rapidly once recording begins.

- To stop a recording early:** type `stop` in the serial monitor. The firmware will upload however many samples were already captured.

- Collect at least **10 windows per label** for a usable dataset
- Keep the sensor orientation consistent across all recordings of the same activity
- Perform the activity naturally — the model learns from real motion patterns
- If an upload fails, check the serial monitor for the error message

Each screen tells you exactly what the device is doing and what to do next.

IDLE	WiFi:OK	← state + WiFi status (inverted bar)
walking		← active label (large text, centered)
WiFi: 192.168.1.42		← IP address
Btn: cycle label		← press button to change label
Serial: 'start' cmd		← type 'start' in serial monitor to record

What to do: Press the button briefly to change the label. Then either hold the button for 2 s or type **start** in the serial monitor to begin recording.

REC	walking	← recording indicator + current label
		← progress bar (fills as samples are collected)
Samples: 350 / 500		← samples so far / total target
Time left: 2s		← countdown

Short: stop & upload

← you can stop early if needed

What to do: Perform the activity naturally. The device stops automatically at 5 s.

UPLOADING — Sending Data

UPLOADING
Label: walking Samples: 500 Sending data to Edge Impulse...

← centered state label

What to do: Wait — this takes 2–10 seconds depending on your WiFi speed.

UPLOAD OK — Success

UPLOAD OK
Label: walking Samples: 500 Sent to Edge Impulse! Returning to IDLE...

← success header

What to do: Nothing — the device returns to IDLE automatically after 2.5 s.

UPLOAD FAILED — Error

UPLOAD FAILED
Label: walking Samples: 500 ERR_HTTP

← failure header

← error code

See serial monitor

What to do: Open the serial monitor for the full error message (see Troubleshooting).

ERROR — Hardware Problem

!! SYSTEM ERROR !!

IMU not detected
Check: SDA→D4, SCL→D5

Check wiring + reset

← inverted error header

← error description

What to do: Check wiring, then press **RESET** on the XIAO.

Serial Monitor Commands

Open the serial monitor (115200 baud) to control the device from your computer. Type a command and press **Enter**.

Command	What it does
label:walking	Set the active label (standing, walking, running, falling)
start	Begin a 5-second recording (must be in IDLE)
stop	Stop the current recording early and upload
status	Print current state, label, and WiFi info
selftest	Read 10 IMU samples — acceleration should be ~9.81 m/s ²
imuconfig	Print all MPU6050 register values (for debugging)
help	List all available commands

Example Session

```
> label:running
[INF][Collector] Label set to "running"

> start
[INF][Collector] Recording started — label: "running"  duration: 5s  rate: 100Hz
[INF][Collector]  [running] 100 samples — 4s remaining
```

```
[INF][Collector]    [running] 200 samples – 3s remaining
...
[INF][Collector] Collection complete – 500 samples captured
[INF][Collector] Uploading to Edge Impulse (label: "running")...
[INF][Collector] Upload complete
```

LED Status Guide

The onboard LED gives instant feedback without needing to watch the screen.

LED Pattern	Meaning
Single 50 ms pulse every 3 s	IDLE — device is alive and waiting for a command
Brief flash on button press	Label cycled — the OLED will confirm the new label
Rapid on/off (every 5 samples)	RECORDING — collecting data
Solid ON	UPLOADING — sending to Edge Impulse
3 slow blinks	Upload succeeded
6 rapid blinks	Upload failed — check serial monitor
Continuous rapid blink (on boot)	Fatal error — IMU not detected

Troubleshooting

IMU Not Detected

```
[ERR][IMUSensor] WHO_AM_I mismatch: expected 0x68, got 0xFF
```

Check	How to fix
SDA/SCL wiring	D4 = SDA, D5 = SCL on the XIAO
Power	MPU6050 VCC must be 3.3 V (not 5 V)
AD0 pin	Connect AD0 to GND to select I2C address 0x68
Cable quality	Try shorter wires; add 4.7 kΩ pull-ups on SDA/SCL if signal is noisy

WiFi Connection Failed

```
[ERR][Network] Connection failed after 15000 ms
```

Check	How to fix
-------	------------

Check	How to fix
Credentials	Verify <code>WIFI_SSID</code> and <code>WIFI_PASSWORD</code> in <code>Config.h</code>
Frequency band	XIAO supports 2.4 GHz only — disable 5 GHz-only mode on your router
Signal strength	Move the board closer to the router for initial testing

Upload Failed — HTTP Error

```
[ERR][EIClient] Server rejected upload (HTTP 401)
```

HTTP code	Meaning	Fix
401	Wrong API key	Copy the key from Edge Impulse Studio → Dashboard → Keys
413	Payload too large	Reduce <code>COLLECTION_DURATION_S</code> or increase <code>JSON_PAYLOAD_MAX_BYTES</code> in <code>Config.h</code>

Payload Buffer Overflow

```
[ERR][EIClient] Payload buffer overflow! Increase JSON_PAYLOAD_MAX_BYTES in Config.h
```

The buffer size formula is `400 + (MAX_SAMPLES × 72)`. Increase `JSON_PAYLOAD_MAX_BYTES` in `Config.h` if you see this error.

Port Not Found (Windows)

- Install the **CH343** USB-serial driver
- Try a different cable — many USB-C cables are **charge-only**
- Check Device Manager for a yellow warning icon on the COM port

Code Deep Dive

This section is for students who want to understand how the firmware is structured and which C++ concepts appear in each file.

Project Structure

```
plastivatch_collector/  
├─ platformio.ini          # Build configuration: board, framework,  
libraries
```

```

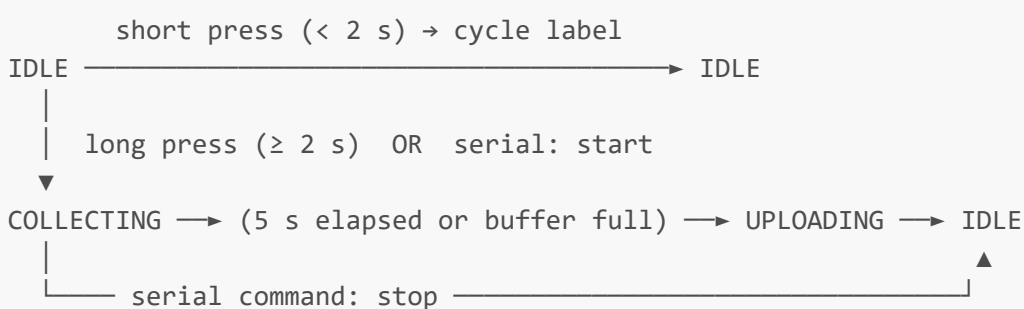
└─ src/
    └─ main.cpp                # Entry point – setup() and loop()
    └─ config/
        └─ Config.h            # ALL constants live here; edit before flashing
    └─ core/
        └─ Logger.h            # Leveled serial logger (header-only, static)
        └─ SampleBuffer.h      # Fixed-capacity IMU sample storage (header-only)
    └─ imu/
        └─ IMUSensor.h         # MPU6050 driver interface
        └─ IMUSensor.cpp        # MPU6050 driver – reads raw I2C bytes → physical
units
└─ network/
    └─ NetworkManager.h        # WiFi manager interface
    └─ NetworkManager.cpp      # WiFi connect / reconnect logic
└─ uploader/
    └─ EdgeImpulseClient.h     # HTTPS uploader interface
    └─ EdgeImpulseClient.cpp   # Builds JSON payload, POSTs to Edge Impulse
└─ display/
    └─ OLEDDisplay.h           # SSD1306 display driver interface
    └─ OLEDDisplay.cpp         # One draw method per application state
└─ collector/
    └─ DataCollector.h         # State machine interface
    └─ DataCollector.cpp       # FSM: IDLE → COLLECTING → UPLOADING → IDLE

```

Key design rule: every file reads constants only from `Config.h`. To change a setting, you edit exactly one file.

How the Application Works: State Machine

`DataCollector` is a **Finite State Machine (FSM)** — the firmware is always in exactly one of these states:



`DataCollector::update()` is called on every `loop()` iteration. It:

1. Polls the label button with a two-stage debouncer (raw edge → stable edge)
 2. Reads and processes any serial command
 3. Dispatches to the correct state handler
-

Module Descriptions

`src/config/Config.h` — Central Configuration

One header of `#define` constants — the only file students need to edit before flashing.

Key groups of settings:

- **WiFi:** SSID, password, connection timeout
- **Edge Impulse:** API key, device name, ingestion endpoint
- **MPU6050:** I2C pins, full-scale ranges, digital filter, sample rate
- **Collection:** recording duration (5 s), window size (500 samples)
- **Display:** OLED I2C address, screen dimensions, result dwell time

`src/core/Logger.h` — Serial Logger (header-only)

A static-only utility class that formats log messages with a level tag and a module tag:

```
[INF][Collector      ] Recording started – label: "walking"
[ERR][IMUSensor     ] WHO_AM_I mismatch: expected 0x68, got 0xFF
[DBG][EIClient      ] Payload built: 36412 bytes
```

Log levels: `DEBUG < INFO < WARNING < ERROR < NONE`

Verbose debug logging is enabled by building with `-DPLASTIWATCH_DEBUG=1` (the `seed_xiao_esp32c3_debug` environment does this automatically).

`src/core/SampleBuffer.h` — IMU Sample Storage (header-only)

A fixed-size array wrapper for one collection window. Uses a **statically allocated** array — no heap allocation, no `std::vector`.

```
struct IMUSample {
    float accX, accY, accZ; // Acceleration [m/s²]
    float gyrX, gyrY, gyrZ; // Angular velocity [deg/s]
};
```

Key methods: `push()`, `count()`, `isFull()`, `isEmpty()`, `clear()`, `operator[]`, `fillRatio()`, `intervalMs()`.

`src/imu/IMUSensor.h/.cpp` — MPU6050 Driver

Communicates with the MPU6050 directly over the Arduino `Wire` (I2C) library — no third-party sensor library required.

Initialization sequence (`begin()`):

1. Start I2C bus at 400 kHz
2. Read `WHO_AM_I` register — must return `0x68` (identity check)
3. Wake the device, select stable PLL clock source

4. Configure full-scale ranges and digital low-pass filter (DLPF)
5. Pre-compute float scale factors for fast conversion in the read loop

Reading data (`readSample()`):

- Burst-reads 14 bytes from register `0x3B` in one I2C transaction
- Reassembles big-endian 16-bit integers using a local lambda
- Multiplies by scale factors to produce m/s^2 and deg/s

Scale factor formulas:

```

accelScale = 9.80665 / (16384 / FS_G)      // raw ADC → m/s²
gyroScale  = 1.0 / (131 / (FS_DPS / 250)) // raw ADC → deg/s

```

`src/network/NetworkManager.h/.cpp` — WiFi Manager

A static-only class (cannot be instantiated) wrapping the ESP32 WiFi library.

Method	Behaviour
<code>connect()</code>	Blocking connect with progress dots; times out after 15 s
<code>ensureConnected()</code>	Non-blocking guard; retries at most once per 5 s
<code>isConnected()</code>	True when an IP address is assigned
<code>localIP()</code> / <code>rssi()</code>	Diagnostics — IP address and signal strength

`src/uploader/EdgeImpulseClient.h/.cpp` — HTTPS Uploader

Serialises a `SampleBuffer` into Edge Impulse's **Data Acquisition JSON** format and POSTs it over HTTPS.

Payload skeleton:

```

{
  "protected": { "ver": "v1", "alg": "none", "iat": 0 },
  "signature": "0000...0000",
  "payload": {
    "device_name": "plastiwatch-001",
    "interval_ms": 10.0,
    "sensors": [
      { "name": "accX", "units": "m/s2" }, ...
    ],
    "values": [
      [-0.12, 9.78, 0.05, 1.20, -0.30, 0.10],
      ...500 rows total...
    ]
  }
}

```

The internal `PayloadBuilder` class writes into a pre-allocated `char[]` buffer with overflow detection — no `std::string`, no dynamic resizing.

Upload result codes:

Code	Meaning
OK	Success (HTTP 200/201)
ERR_NO_WIFI	Not connected to WiFi
ERR_NO_SAMPLES	Buffer is empty
ERR_ALLOC	Heap allocation failed
ERR_OVERFLOW	JSON exceeded buffer size
ERR_HTTP	Server returned a non-2xx response
ERR_TIMEOUT	Connection or request timed out

src/display/OLEDDisplay.h/.cpp — OLED Driver

Wraps the Adafruit SSD1306 library. Exposes one draw method per application state. All methods are safe no-ops if the display is not connected (`begin()` returned false).

Private helpers:

- `printCentered(text, y, size)` — centres text horizontally at a given y coordinate
- `drawHeaderBar(left, right)` — draws the inverted (white-fill, black-text) header bar

src/collector/DataCollector.h/.cpp — State Machine

The application's core logic. Orchestrates IMU reading, button input, serial commands, display updates, and upload sequencing.

update() is called every loop iteration and:

1. Polls the label button (GPIO3) with a two-stage debouncer
2. Polls the serial port for text commands
3. Dispatches to `handleIdle()`, `handleCollecting()`, or `handleUploading()`

Button behaviour (IDLE state only):

- Short press** (`< BTN_LONG_PRESS_MS = 2000 ms`) → cycle the active label
- Long press** (`≥ 2 s`, then release) → start recording (equivalent to serial `start`)
- The debouncer ignores transitions shorter than `BTN_DEBOUNCE_MS` (50 ms), preventing contact bounce from registering as multiple presses
- All actions fire on **button release** (rising edge); press-start time is stamped on the falling edge to measure hold duration

Starting / stopping a recording: button long-press, or serial commands `start` / `stop`

`src/main.cpp` — Entry Point

Standard Arduino structure, kept intentionally minimal:

```
setup()
|
|─ Serial.begin(115200)           set up serial monitor
|─ Logger::setLevel(...)         INFO or DEBUG depending on build flags
|─ display.begin()               initialise OLED (safe to skip if not attached)
|─ imu.begin()                   initialise MPU6050 – HALT on failure
|─ NetworkManager::connect()     connect to WiFi – warn and continue offline
|─ collector.begin()             show initial IDLE screen

loop()
└─ collector.update()            non-blocking, returns immediately every
iteration
```

C++ Concepts Used

This project was designed as a teaching tool. Each concept is used deliberately and appears in the same place every time.

Concept	Where Applied	Why
<code>enum class</code> (scoped enum)	<code>LogLevel</code> , <code>CollectorState</code> , <code>UploadResult</code>	Type-safe; can't accidentally compare states to integers
Static-only / utility classes	<code>Logger</code> , <code>NetworkManager</code> , <code>EdgeImpulseClient</code>	No instance needed — constructor is <code>deleted</code>
Constructor member initializer lists	<code>DataCollector</code> , <code>IMUSensor</code>	Preferred C++ initialization style
<code>constexpr</code>	<code>SampleBuffer::CAPACITY</code> , IMU register addresses	Values resolved at compile time — zero runtime cost
<code>explicit</code> constructor	<code>DataCollector(IMUSensor& imu)</code>	Prevents accidental implicit conversions
Reference member	<code>IMUSensor& m_imu</code> in <code>DataCollector</code>	Dependency injection without ownership
Lambda	<code>toInt16</code> in <code>readSample()</code>	Short, inline helper that captures nothing
Anonymous namespace	<code>PayloadBuilder</code> in <code>EdgeImpulseClient.cpp</code>	Hides implementation details from the linker

Concept	Where Applied	Why
<code>new (std::nothrow)</code>	Heap allocation in <code>EdgeImpulseClient</code>	Returns <code>nullptr</code> on failure instead of throwing
<code>static</code> local variables	<code>lastHeartbeatMs</code> / <code>ledOn</code> in <code>handleIdle()</code>	Persist across function calls without being global
<code>#pragma once</code>	All header files	Include guard — simpler than <code>#ifndef</code> guards
<code>inline static</code> member	<code>Logger::s_level</code>	C++17 feature — lets you define a static member inside a header
Variadic functions (<code>va_list</code>)	<code>Logger::infof()</code> , <code>PayloadBuilder::appendf()</code>	Printf-style formatting with variable argument counts
<code>reinterpret_cast</code>	<code>char* → uint8_t*</code> in <code>EdgeImpulseClient</code>	Type-safe cast required by the HTTPClient API
<code>static_cast</code>	Numeric conversions throughout	Explicit, readable type conversions
Operator overloading	<code>SampleBuffer::operator[]</code>	Natural array-style element access
<code>switch</code> on scoped enum	State dispatch in <code>update()</code>	Clean, exhaustive state handling

Build Configuration (`platformio.ini`)

```
[env:seeed_xiao_esp32c3]
board            = seeed_xiao_esp32c3
platform        = espressif32
framework       = arduino
upload_speed    = 921600
build_flags     = -std=gnu++17           ← enables C++17 (inline static members, etc.)
lib_deps =
  adafruit/Adafruit SSD1306@^2.5.9
  adafruit/Adafruit GFX Library@^1.11.9
```

PlatformIO downloads all library dependencies automatically on the first build — no manual library installation required.

Target board: Seeed Studio XIAO ESP32C3 — Framework: Arduino — Build system: PlatformIO